Algoritmos

2026-04-01

Ejercicio 1 de algoritmo

Consigna: Las últimas 3 crifras de mi DNI son 994, consigna crear una variable que tenga ese número

```
DNI=994
```

Consigna: Crear un vector que tenga todos los números enteros desde 1 al 994.

```
secuencia_dni<-(1:994)
secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 994
```

Ejercicio 2

calcular la suma de todos los valores del vector secuencia_dni

```
total<-0
valor_final<-length(secuencia_dni)
for (i in 1:valor_final)
  total<-total+i
total
```

```
## [1] 494515
```

Ejercicio 3

Repetir el ejercicio anterior pero en Phyton

```
suma = 0

for i in range(1, 994):
    suma += i

print(suma)
```

```
## 493521
```

Ejercicio 4

¿Cuanto tarda en correr el código del ejercicio 2?

```
system.time({
  total <- 0
  valor_final <- length(secuencia_dni)

  for (i in 1:valor_final) {
    total <- total + i
  }

  print(total)
})
```

```
## [1] 494515
```

```
##    user  system elapsed
##   0.002   0.000   0.002
```

Ejercicio 5

Repetir el ejercicio anterior utilizando la cabeza de Gauss

```
system.time({
  n <- length(secuencia_dni)

  # Aplicamos la fórmula de Gauss
  total_gauss <- (n * (n + 1)) / 2

  print(total_gauss)
})
```

```
## [1] 494515
```

```
##    user  system elapsed
##   0.000   0.000   0.001
```

Ejercicios del profe:

1.4 Generar un vector secuencia

```
# --- 1. Generar secuencia con bucle 'for' ---
# Primero inicializamos el vector
```

```r
system.time({
  A <- 0
  for (i in 1:50000) {
    A[i] <- (i * 2)
  }
})
```

```
##    user  system elapsed
##   0.008   0.004   0.012
```

```r
# Verificamos los resultados
head(A)
```

```
## [1]  2  4  6  8 10 12
```

```r
tail(A)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

```r
system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})
```

```
##    user  system elapsed
##   0.001   0.000   0.000
```

```r
# Verificamos los resultados
head(B)
```

```
## [1]  2  4  6  8 10 12
```

```r
tail(B)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

1.5 Implementación de una serie Fibonachi o Fibonacci

```r
# Definimos la cantidad de elementos de la serie que queremos calcular
n <- 30

# --- Método 1: Bucle 'for' con pre-asignación
system.time({
  fib <- numeric(n)
  fib[1] <- 0
  fib[2] <- 1

  for (i in 3:n) {
    fib[i] <- fib[i-1] + fib[i-2]
  }
})
```

```
##    user  system elapsed
##   0.003   0.000   0.003
```

```r
# Ver los primeros resultados
print("Primeros números de la serie:")
```

```
## [1] "Primeros números de la serie:"
```

```r
head(fib, 10)
```

```
##  [1]  0  1  1  2  3  5  8 13 21 34
```

```r
# --- Método 2: Función Recursiva

fib_recursiva <- function(k) {
  if (k == 0) return(0)
  if (k == 1) return(1)
  return(fib_recursiva(k - 1) + fib_recursiva(k - 2))
}

print("Tiempo ejecución recursiva:")
```

```
## [1] "Tiempo ejecución recursiva:"
```

```r
system.time({
  # Calculamos el elemento n de forma individual
  resultado_rec <- fib_recursiva(n)
})
```

```
##    user  system elapsed
##   0.763   0.000   0.768
```

1.6 De nición matemática recurrente

```r
# --- Resolver consigna con bucle 'while' ---

# Inicializamos la serie según la definición matemática
fib <- c(0, 1)
i <- 2 # Empezamos el conteo de iteraciones/posiciones

system.time({
  # El bucle sigue mientras el último número generado sea <= 1.000.000
  while (fib[i] <= 1000000) {
    i <- i + 1
    # Definición recurrente: f(n+1) = f(n) + f(n-1)
    fib[i] <- fib[i-1] + fib[i-2]
  }
})
```

```
##    user  system elapsed
##   0.003   0.000   0.003
```

```r
# --- Resultados ---
cat("El primer número mayor a 1.000.000 es:", fib[i], "\n")
```

```
## El primer número mayor a 1.000.000 es: 1346269
```

```r
cat("Se necesitaron", i - 1, "iteraciones (pasos) para superarlo.\n")
```

```
## Se necesitaron 31 iteraciones (pasos) para superarlo.
```

```r
# Mostramos los últimos elementos para verificar
tail(fib)
```

```
## [1]  121393  196418  317811  514229  832040 1346269
```

1.7 Ordenacion de un vector por metodo burbuja

```r
# --- 1. Creamos un vector desordenado ---
# Usamos un vector de 1000 números aleatorios
set.seed(123) # Para que el resultado sea siempre el mismo
vector_desordenado <- sample(1:1000, 1000)

# --- 2. Implementación del Método Burbuja ---
bubble_sort <- function(x) {
  n <- length(x)
  # El bucle externo recorre todo el vector
  for (i in 1:(n - 1)) {
    # El bucle interno compara pares adyacentes
    for (j in 1:(n - i)) {
      if (x[j] > x[j + 1]) {
        # Intercambio de posición (el "burbujeo")
        temp <- x[j]
        x[j] <- x[j + 1]
        x[j + 1] <- temp
      }
    }
  }
  return(x)
}

# --- 3. Medimos Performance del Burbuja vs R Nativo ---

cat("Tiempo con Método Burbuja:\n")
```

```
## Tiempo con Método Burbuja:
```

```r
system.time({
  vector_ordenado_burbuja <- bubble_sort(vector_desordenado)
})
```

```
##    user  system elapsed
##   0.058   0.000   0.057
```

```r
cat("\nTiempo con función sort() nativa de R:\n")
```

```
##
## Tiempo con función sort() nativa de R:
```

```r
system.time({
  vector_ordenado_R <- sort(vector_desordenado)
})
```

```
##    user  system elapsed
##   0.000   0.000   0.001
```

1.8 La penitencia de Newton

```r
# Definimos el límite de la suma (1*10^7 para que no tarde demasiado en R)
# Nota: 1*10^10 es un número extremadamente grande para un bucle for en R
n <- 1e7
```

```r
# --- MÉTODO 1: La "Penitencia" (Bucle For / Fuerza Bruta) ---
# Este método suma uno por uno, como quería el profesor.
system.time({
  suma_iterativa <- 0
  for (i in 1:n) {
    suma_iterativa <- suma_iterativa + i
  }
})
```

```
##    user  system elapsed
##   0.160   0.000   0.162
```

```r
cat("Resultado Suma Iterativa:", suma_iterativa, "\n")
```

```
## Resultado Suma Iterativa: 5e+13
```

```r
# --- MÉTODO 2: La "Estrategia de Newton/Gauss" (Fórmula Analítica) ---
# Usamos la suma de una progresión aritmética: (n * (n + 1)) / 2
system.time({
  suma_analitica <- (n * (n + 1)) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

```r
cat("Resultado Suma Analítica:", suma_analitica, "\n")
```

```
## Resultado Suma Analítica: 5e+13
```

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

primeros comandos de de R

R es un lenjuage muy parecido al de matlab y trabaja con variables que en general pueden ser matrices.

```r
a=38
 a=40
 b=60
 c=b-a
 c
```

```
## [1] 20
```

```r
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00  
```

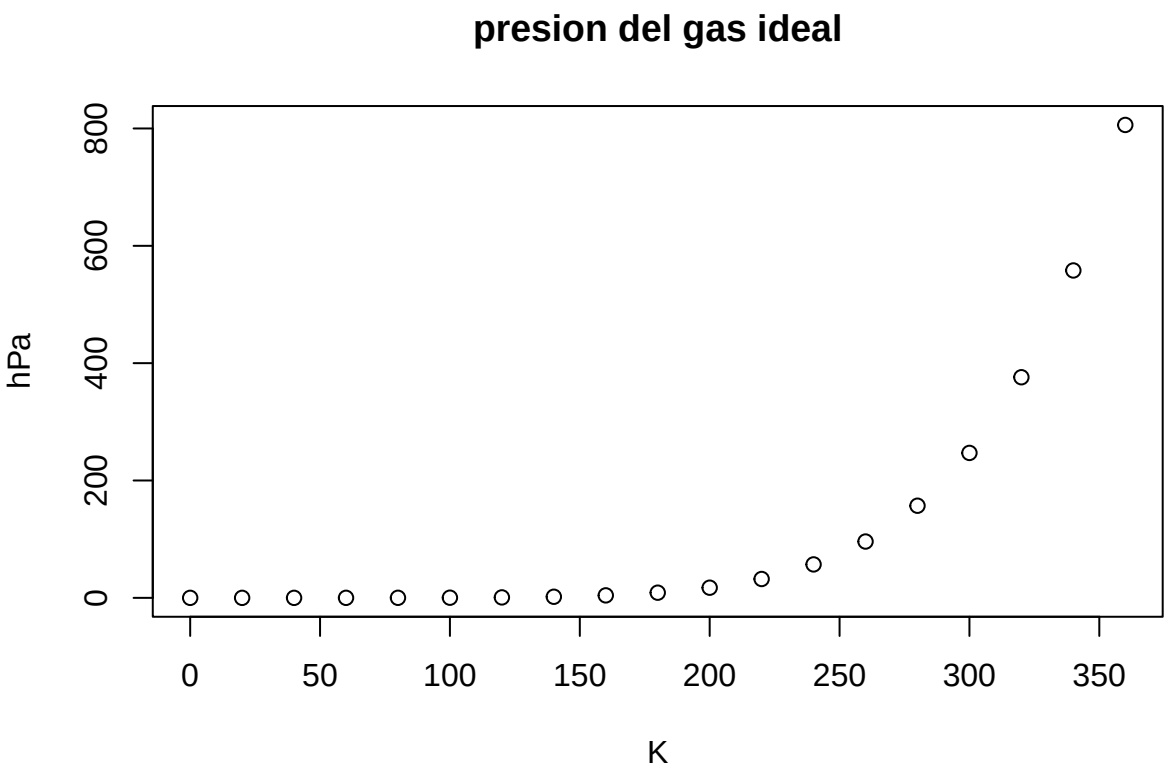
Uso de datasets o base de datos internos de R

Usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos $ despues de el comando de informacion especifico.

Including Plots

You can also embed plots, for example:

```
plot(pressure, main="presion del gas ideal", ylab="hPa", xlab="K")
```



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

##Comando plot Es un comando que permite graficar cualquier fuente de datos que le asignamos, teniendo tambien la posibilidad de asignar nombre a las variables o hacer cambios al grafico. En caso de duda de como utilizar el comando podemos escribir ##plot## en la consola, lo cual abrira una ventana que explicara en detalle el comando.

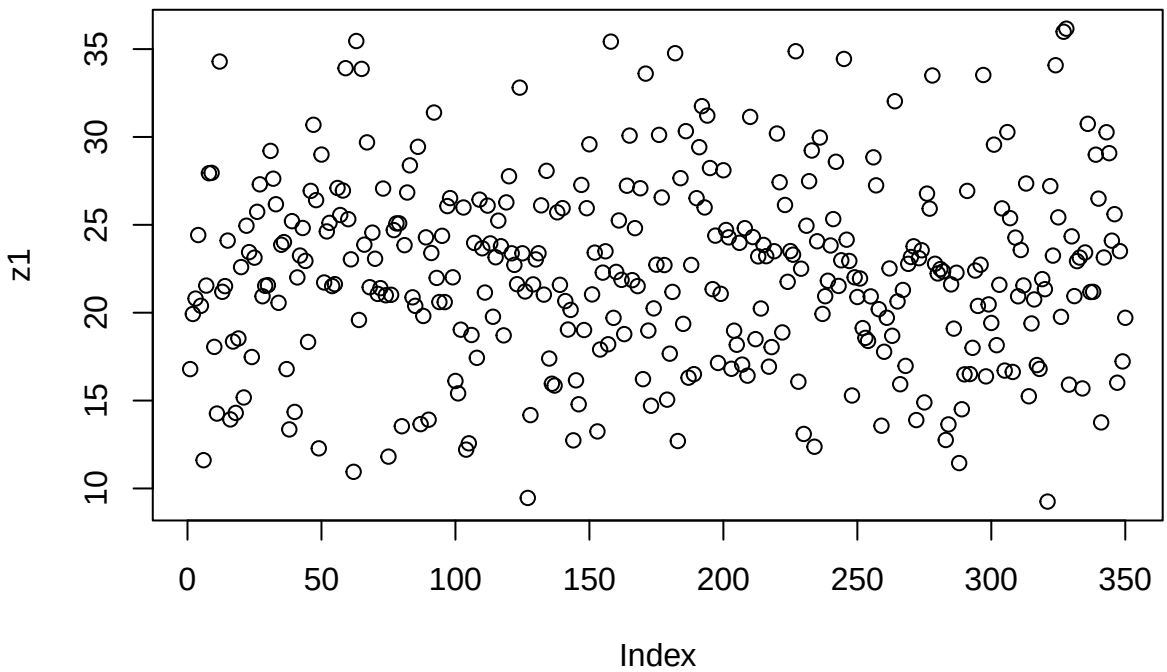
funciones estadisticas

```
z1<-rnorm(350,22,5)
z1
```

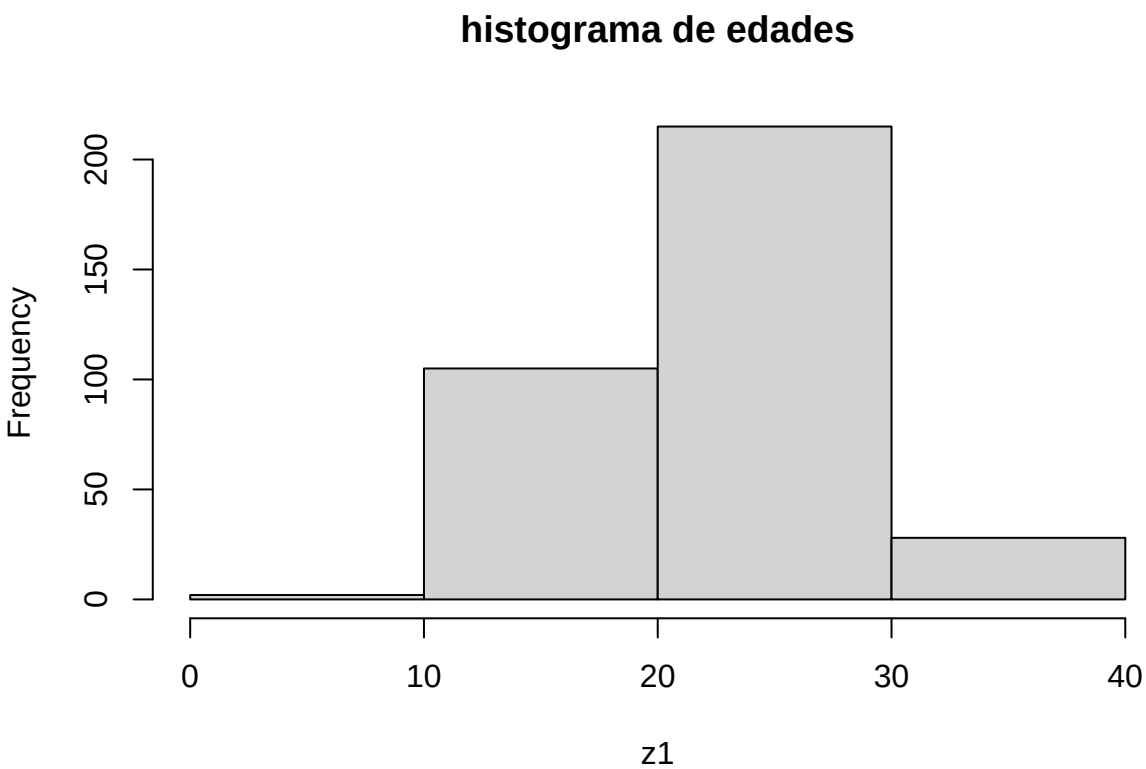
```
##   [1] 16.791556 19.927056 20.804855 24.418088 20.393376 11.607554 21.542829
##   [8] 27.935934 27.958006 18.055184 14.261117 34.290302 21.187890 21.512744
##  [15] 24.102871 13.929803 18.358904 14.297788 18.534527 22.594247 15.176453
##  [22] 24.949913 23.446720 17.478925 23.131625 25.740406 27.305476 20.935759
##  [29] 21.531816 21.566429 29.207309 27.625359 26.172008 20.563296 23.866207
##  [36] 24.016452 16.791634 13.358477 25.209150 14.353447 22.008418 23.251239
##  [43] 24.819337 22.947131 18.335731 26.931829 30.693169 26.405894 12.281745
```

```
##  [50] 28.997881 21.719720 24.624571 25.110166 21.516570 21.623684 27.095785
##  [57] 25.558010 26.951311 33.914633 25.322079 23.036906 10.946834 35.458570
##  [64] 19.586616 33.873674 23.873218 29.692151 21.451448 24.557354 23.069790
##  [71] 21.069397 21.398031 27.064172 20.992709 11.811588 21.020554 24.698953
##  [78] 25.082279 25.082839 13.539492 23.843710 26.839296 28.382893 20.875194
##  [85] 20.390537 29.439189 13.660360 19.815850 24.287310 13.911131 23.398139
##  [92] 31.389320 21.979697 20.607730 24.374559 20.604639 26.067002 26.522177
##  [99] 22.013458 16.116539 15.408896 19.035013 25.986903 12.208974 12.568374
## [106] 18.731101 23.971974 17.432170 26.433745 23.666850 21.146802 26.094141
## [113] 23.941826 19.770325 23.155575 25.237567 23.781417 18.709949 26.276011
## [120] 27.764681 23.381373 22.720523 21.621875 32.807079 23.381578 21.208530
## [127]  9.460411 14.173591 21.611634 23.031470 23.384362 26.107534 21.029238
## [134] 28.072944 17.392420 15.957786 15.855069 25.711485 21.585400 25.949090
## [141] 20.661468 19.040539 20.158237 12.736916 16.151924 14.789827 27.271611
## [148] 19.013350 25.947299 29.582453 21.041126 23.419395 13.244662 17.906651
## [155] 22.281074 23.495435 18.203009 35.424295 19.708049 22.321218 25.248959
## [162] 21.869907 18.782163 27.226528 30.077727 21.851530 24.811337 21.512938
## [169] 27.082276 16.219163 33.604301 18.982344 14.705753 20.245411 22.733542
## [176] 30.118106 26.556048 22.712292 15.052582 17.669811 21.183575 34.765131
## [183] 12.698862 27.655273 19.363829 30.329954 16.303997 22.718116 16.502245
## [190] 26.517582 29.418897 31.753605 25.988003 31.216331 28.232120 21.340625
## [197] 24.385186 17.140029 21.073989 28.104819 24.706421 24.286787 16.809345
## [204] 18.977434 18.176970 23.976479 17.047462 24.810207 16.417918 31.142652
## [211] 24.302957 18.494982 23.205230 20.237734 23.855740 23.217664 16.929429
## [218] 18.043431 23.497968 30.195260 27.423085 18.877163 26.129615 21.757158
## [225] 23.506568 23.301807 34.877249 16.073556 22.504599 13.100114 24.949180
## [232] 27.483042 29.228311 12.374274 24.063847 29.966850 19.929921 20.939247
## [239] 21.817314 23.825094 25.325799 28.589104 21.522562 22.981390 34.439989
## [246] 24.155495 22.943766 15.288784 22.014280 20.893369 21.944771 19.122912
## [253] 18.565922 18.396132 20.927477 28.840663 27.245433 20.200124 13.570418
## [260] 17.777083 19.711197 22.518190 18.686964 32.033403 20.638662 15.930278
## [267] 21.293691 16.973112 22.780779 23.168168 23.777938 13.890709 23.103556
## [274] 23.552250 14.894458 26.776828 25.920854 33.498097 22.783515 22.233668
## [281] 22.482929 22.348831 12.757636 13.644365 21.612305 19.094663 22.273683
## [288] 11.443958 14.506509 16.492583 26.930291 16.507550 18.002430 22.399369
## [295] 20.386268 22.732086 33.525310 16.376982 20.472652 19.416203 29.561977
## [302] 18.152575 21.589565 25.935668 16.707047 30.275879 25.378812 16.628967
## [309] 24.272889 20.933464 23.566144 21.550124 27.352580 15.244498 19.386917
## [316] 20.754047 17.021006 16.800225 21.910099 21.339124  9.253286 27.202867
## [323] 23.248629 34.081037 25.425991 19.765203 35.986956 36.161130 15.906441
## [330] 24.345160 20.943765 22.935256 23.137714 15.690498 23.427948 30.746237
## [337] 21.179550 21.185366 28.992860 26.491981 13.757526 23.142785 30.267736
## [344] 29.076382 24.099758 25.606104 16.015324 23.500658 17.227755 19.709910
```

```r
plot(z1)
```




```r
hist(z1, main="histograma de edades", breaks=2)
```

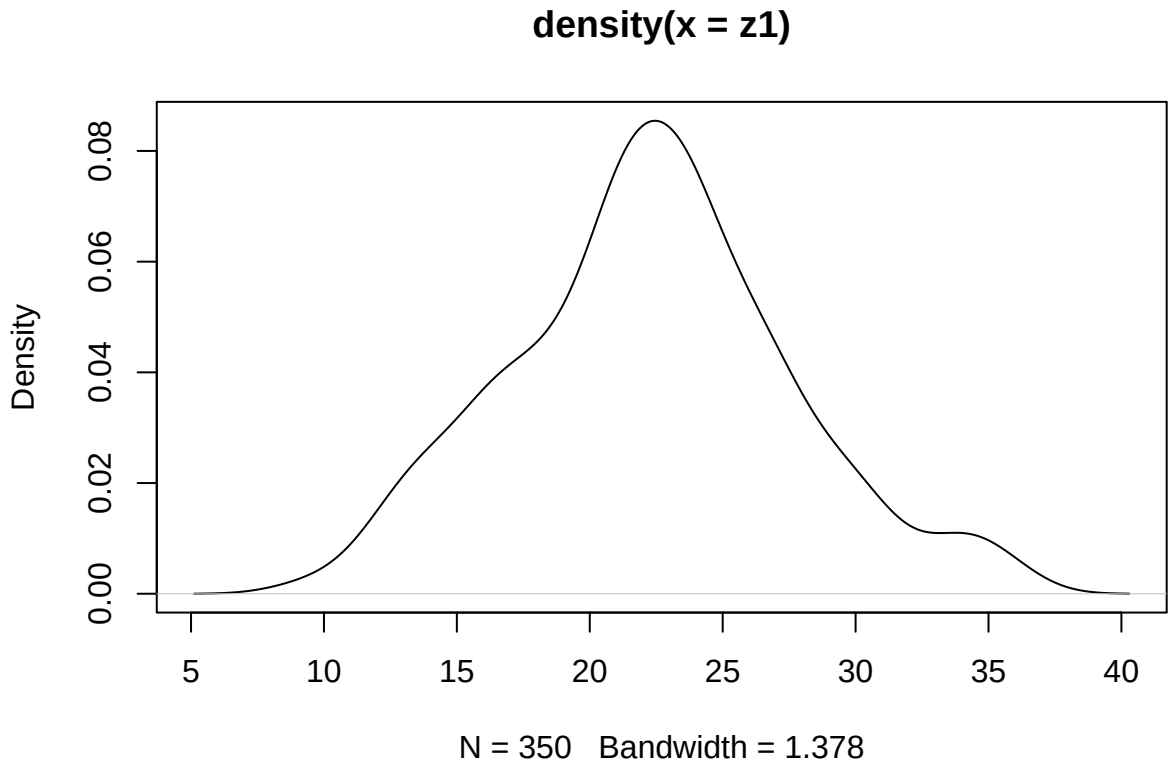



```r
density(z1)
```

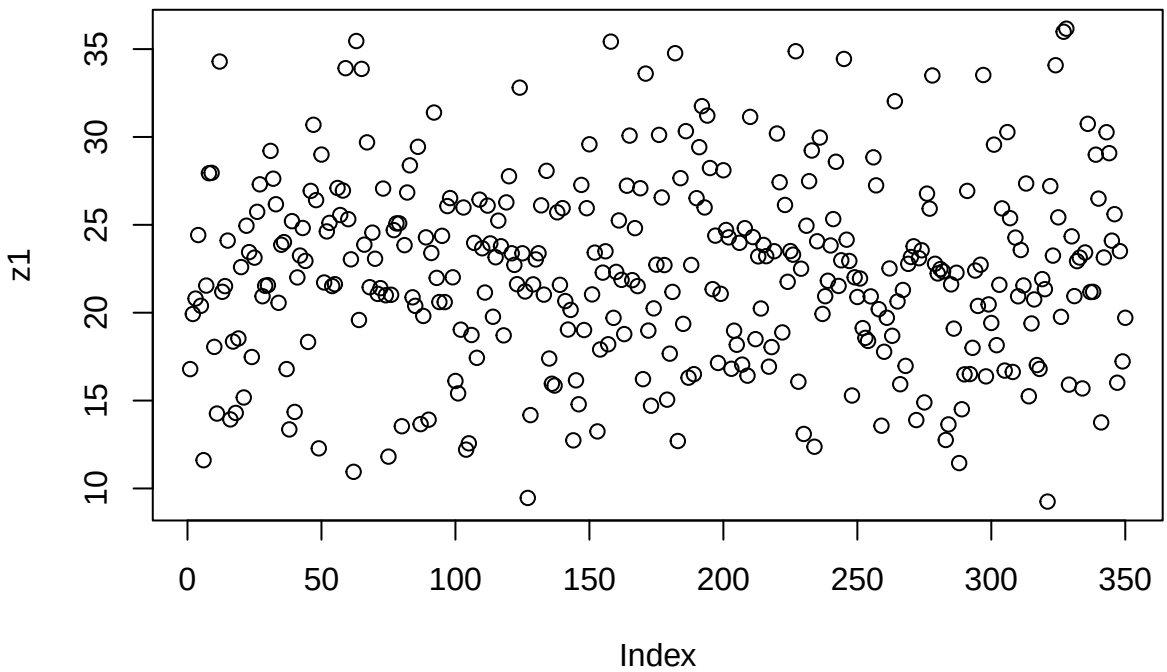
```
##
## Call:
##  density.default(x = z1)
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.378
```

```
##
##        x                y
##  Min.   : 5.118   Min.   :1.517e-05
##  1st Qu.:13.913   1st Qu.:4.994e-03
##  Median :22.707   Median :2.014e-02
##  Mean   :22.707   Mean   :2.837e-02
##  3rd Qu.:31.502   3rd Qu.:4.585e-02
##  Max.   :40.296   Max.   :8.545e-02
```

```r
plot(density(z1))
```




```r
plot(z1)
```




```r
w1<-length(z1)
w1
```

```
## [1] 350
```

```r
x1<-(500:849)
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
# 2.0 Comparación de Performance con Gráficos de Violín
# Instalamos y cargamos la librería necesaria
if(!require(microbenchmark)) install.packages("microbenchmark")
```

```
## Loading required package: microbenchmark
```

```r
if(!require(ggplot2)) install.packages("ggplot2")
```

```
## Loading required package: ggplot2

## Warning in library(package, lib.loc = lib.loc, character.only = TRUE,
## logical.return = TRUE, : there is no package called 'ggplot2'

## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.5'
## (as 'lib' is unspecified)

## also installing the dependencies 'cpp11', 'farver', 'labeling', 'RColorBrewer', 'viridisLite', 'gtabl
```

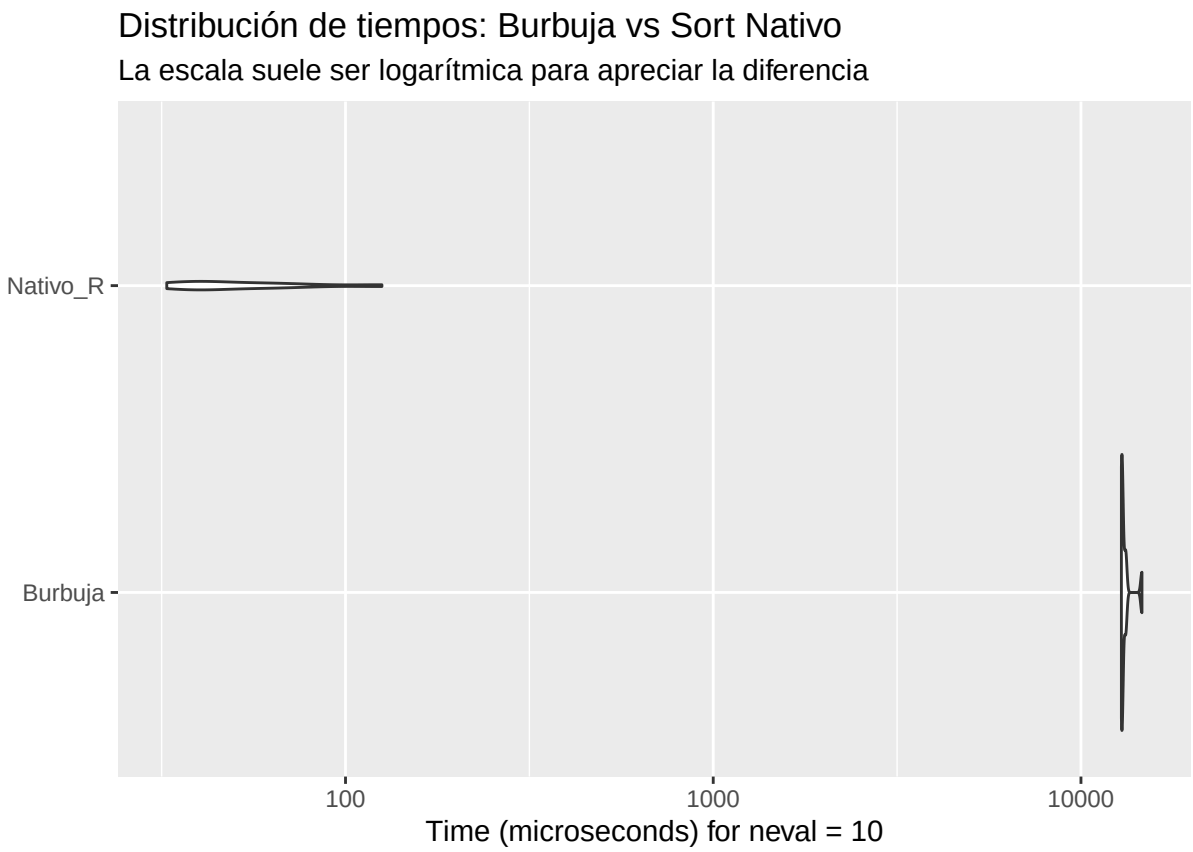
```r
library(microbenchmark)
library(ggplot2)

# --- 2.1 Comparación: Burbuja vs Sort Nativo ---
set.seed(123)
vector_prueba <- sample(1:500, 500)

# Usamos microbenchmark para ejecutar los métodos varias veces y comparar
comparativa_sort <- microbenchmark(
  Burbuja = bubble_sort(vector_prueba),
  Nativo_R = sort(vector_prueba),
  times = 10 # Lo corremos 10 veces para tener una muestra estadística
)

# Graficamos el comportamiento (Gráfico de Violín)
autoplot(comparativa_sort) +
  labs(title = "Distribución de tiempos: Burbuja vs Sort Nativo",
       subtitle = "La escala suele ser logarítmica para apreciar la diferencia",
       y = "Tiempo de ejecución", x = "Algoritmo")
```

```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the microbenchmark package.
##   Please report the issue at
##   <https://github.com/joshuaulrich/microbenchmark/issues/>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

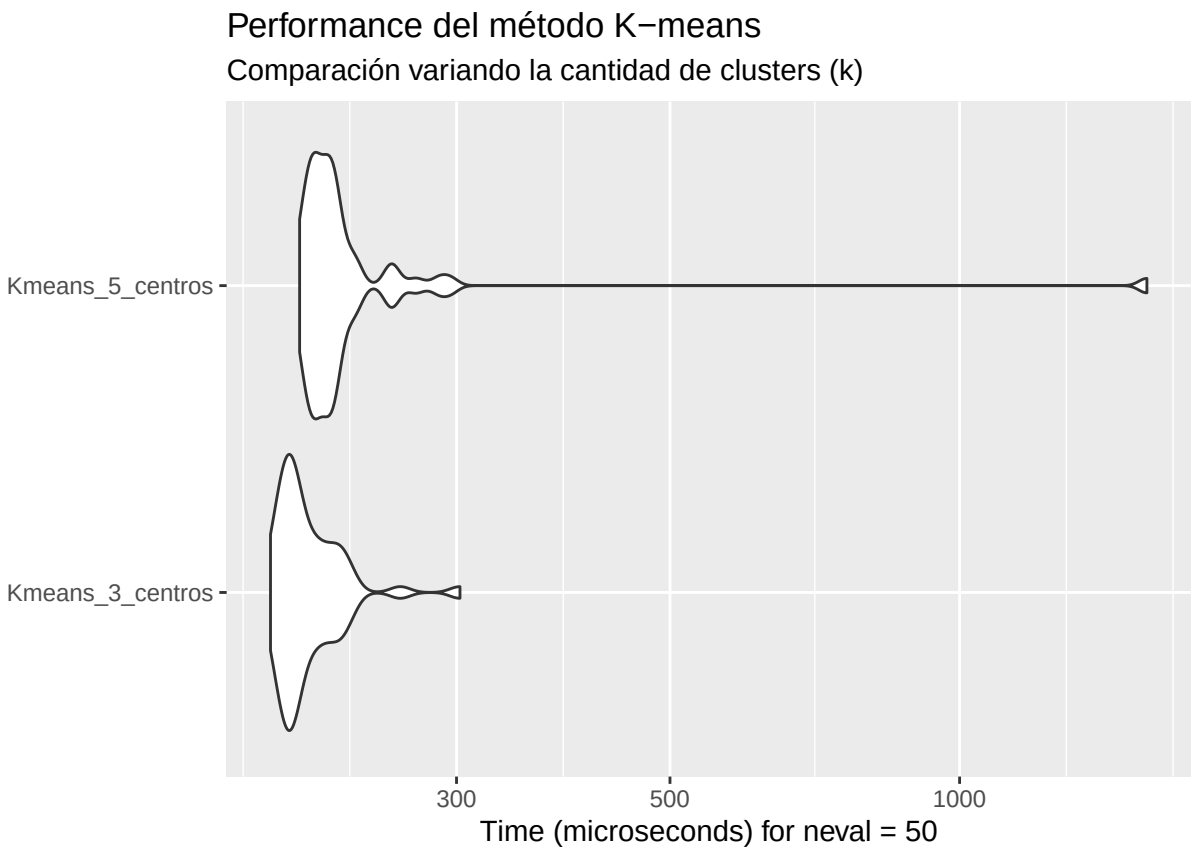



```r
# --- 2.2 K-means: Introducción Teórica y Medición ---
```

Introducción Teórica a K-means: El algoritmo de K-means es un método de agrupamiento (clustering) que particiona un conjunto de n observaciones en k grupos. Funciona de forma iterativa asignando cada punto al "centroide" más cercano y recalculando dichos centros hasta que las posiciones se estabilizan. Es ampliamente utilizado en minería de datos para encontrar patrones no visibles a simple vista.

```r
# Medimos el rendimiento de kmeans con un dataset interno de R (iris)
comparativa_kmeans <- microbenchmark(
  Kmeans_3_centros = kmeans(iris[, 1:4], centers = 3),
  Kmeans_5_centros = kmeans(iris[, 1:4], centers = 5),
  times = 50
)

# Graficamos los resultados de K-means
autoplot(comparativa_kmeans) +
  labs(title = "Performance del método K-means",
       subtitle = "Comparación variando la cantidad de clusters (k)",
       y = "Tiempo de ejecución", x = "Configuración")
```


Performance del método K−means
Comparación variando la cantidad de clusters (k)
Kmeans_5_centros
Kmeans_3_centros
300
500
1000
Time (microseconds) for neval = 50